

Satpuda Core Accounting Logic Audit and Rebuild Guide

Read-only review of Sales, Sales History, Purchase, Purchase History, GST, discount, due, credit, paid amount, profit, account-cleared logic, and future return support. Generated April 25, 2026.

Important: application code was not changed for this audit. This PDF is the generated deliverable.

1. Scope and Files Reviewed

The review traced calculation and data flow through the Python/Tkinter screens and database schema. It also performed one read-only check of veterinary.db to confirm the live schema, triggers, row counts, and GST storage signals.

- Sales entry: ui/billing.py
- Sales history and bill actions: ui/sales_history.py
- Sales edit/print helpers used by history: widgets/bill_edit.py and widgets/bill_preview.py
- Purchase entry: ui/purchase.py
- Purchase history and purchase edit/delete/report actions: ui/purchase_history.py
- Schema and trigger setup: main.py
- Customer due cache: core/customer_service.py

2. Live Database Signals

The database was opened read-only. This confirms the deployed schema contains the expected accounting columns and four triggers: trg_sales_after_insert, trg_sales_after_update, trg_purchases_after_insert, and trg_purchases_after_update.

Area	Read-only finding
Current counts	sales: 28, sales_items: 57, purchases: 87, purchase_items: 157
Sales GST storage	sales_items with saved GST > 0: 0
Current sold medicines with GST	sales items whose current medicine record has GST > 0: 21
Purchase credits	purchases with credit_amount > 0: 0, purchases with previous_credit > 0: 0
Sales credits	sales with credit_amount > 0: 1, sales with previous_credit > 0: 0

The GST signal is important: sale item GST is not historically stored even though some sold medicines currently have GST. Any future filing/report based on sales_items.gst_percent will miss sales GST unless this is rebuilt or backfilled from a reliable historical source.

3. Database Model Currently Used

Table	Accounting fields currently present
sales	total_amount, discount, rounding, amount_paid, cash_paid, online_paid, previous_due, previous_credit, total_due, due_amount, credit_amount, paid_due, bill_cleared, account_cleared
sales_items	qty, rate, gst_percent, amount. No item discount field, no taxable value, no GST amount, no cost snapshot.
purchases	total_amount, gst_calc_method, overall_discount, rounding, amount_paid, previous_due, previous_credit, total_due, due_amount, credit_amount, paid_due, bill_cleared, account_cleared
purchase_items	qty, free_qty, hsn_code, gst_value, mrp, rate, amount, discount_percent
customers	cached total_due and total_credit are updated from the latest sales row

The model mixes invoice totals, running party balances, credit, and UI status flags in the invoice tables themselves. That makes edits, deletes, returns, and GST/profit reports fragile because each historical row depends on previous and later rows.

4. Sales Entry Flow

4.1 Item Amount

- For tablets/bolus, ui/billing.py calculates sale rate as MRP divided by units per stripe. For other products, sale rate is MRP.
- Item base amount = quantity * sale rate.
- Medicine/item discount is a percentage typed in the item row. The amount added to selected_medicines is already reduced:
discounted amount = base amount * (1 - medicine_discount / 100).
- The medicine discount percentage is not saved to sales_items. Only the reduced amount is saved.

4.2 Bill Discount, Rounding, Total Amount

- Bill discount entry is treated as a percentage in the new sales screen.
- discount_amount = subtotal * discount_percent / 100.
- pre_round_total = subtotal - discount_amount.
- Rounding is auto-filled to nearest rupee when the rounding entry is not focused.
- total_after_discount = subtotal - discount_amount + rounding.
- sales.total_amount stores the final total after discount and rounding. sales.discount stores the rupee discount amount, not the discount percentage.

4.3 Paid, Due, Credit, Account Clear

- amount_paid = cash_paid + online_paid.
- previous_due is displayed from customer cache. If a customer has credit, the UI stores it internally as negative previous_due, but generate_bill later uses max(0, previous_due), so previous credit does not offset the next sale.
- If amount_paid is less than current total, due_amount is current bill unpaid amount and total_due = previous_due + due_amount.
- If amount_paid is more than current total, excess first clears previous_due, then remaining excess becomes credit_amount.
- bill_cleared = 1 when due_amount is zero. account_cleared = 1 when total_due is zero.
- previous_credit and paid_due columns exist but are not meaningfully used in the new sale save path.

4.4 GST in Sales

- Sales entry does not calculate GST into sale totals. It only displays the selected GST percent as included in MRP.
- When saving sale items, billing inserts gst_percent as 0 instead of the selected medicine GST percent.
- Bill preview calculates included GST from the current medicines.gst_percent value, not from the historical sales_items.gst_percent value.
- Because sales_items.gst_percent is zero in live data, any sales export/report that uses sales_items.gst_percent will show zero GST. Reprinted bills may show GST from the current medicine record, which can change later.

4.5 Sales Entry Risks

Risk	Why it matters
Customer credit is ignored on the next sale	A credit balance is shown as Credit but not applied in generate_bill. The customer can still be charged full amount.
GST is not historical	GST filing cannot rely on sales_items. Reprints can change if medicine GST changes.
Discount input is not stored	Only rupee discount is stored for bill-level discount; item discount is lost.

Risk	Why it matters
Due fields are duplicated by triggers	Application code and database triggers both set clear flags, making behavior harder to reason about.

5. Sales History Flow

5.1 Display and Filters

- Sales History reads stored sales rows directly: total_amount, discount, amount_paid, cash_paid, online_paid, previous_due, due_amount, credit_amount, total_due, bill_cleared, account_cleared.
- Rows are tagged green for account_cleared, yellow for bill_cleared but account not cleared, and red when total_due is positive.
- When account_cleared is 1, the history display forces Total Due to 0, even if a credit exists. Credit is shown separately only in its own column.
- Due Only filter uses total_due > 0 and account_cleared = 0, which is better than filtering only current bill due.

5.2 Summary and Profit

- Total Sales is sum of total_amount over displayed rows.
- Total Discount is sum of sales.discount over displayed rows. Since sales.discount stores rupee amount, this is amount-based, not percentage-based.
- Total Due summary is queried globally from the database as latest uncleared row per customer. It does not fully respect the visible date/customer filters.
- Profit summary uses current medicines.mrp and current medicines.rate joined to historical sales_items. Formula is approximately qty * (current MRP - current rate).
- Profit ignores sale discounts, rounding, GST, historical purchase cost, batch cost, returns, and current-vs-historical price changes. It is not suitable for true profit calculation.

5.3 Bill Details and Printing

- Bill details display sales_items.gst_percent, which is currently saved as 0 by the new sale flow.
- Print Bill uses widgets/bill_preview.py. The preview calculates included GST from current medicine GST instead of saved sale item GST. This hides the storage issue but makes historical bills mutable.

5.4 Sales Edit From History

- Sales History opens widgets/bill_edit.py for edits.
- bill_edit loads sales.discount into the discount entry, but sales.discount is a rupee amount while bill_edit treats the entry as a percentage. Example: original 5 percent discount on Rs.1000 stores Rs.50; editing later loads 50 and treats it as 50 percent.
- When adding a new item during bill edit, bill_edit uses selected_medicine.rate, which is purchase cost, not MRP. It also does not apply the tablet/bolus pack conversion used by the main sales screen.
- bill_edit saves sales_items.gst_percent as 0 again.
- bill_edit uses the frozen previous_due from the original sale row. If earlier bills are edited/deleted, later balances are not recomputed in sequence.

5.5 Sales Delete

- Deleting a sale restores medicine stock and deletes sales_items and sales rows.
- It does not recalculate customer running balances for later sales rows.
- It does not refresh customers.total_due/total_credit after deletion.
- Deleting a historical bill can therefore leave later total_due/account_cleared values stale.

6. Purchase Entry Flow

6.1 Item Amount

- Purchase item base amount = qty * rate. Free quantity affects stock but not purchase amount.
- Item discount is a percentage stored in purchase_items.discount_percent.
- GST value is stored per item in purchase_items.gst_value.
- For discount_before_gst at item level: net price = base * (1 - item_discount/100), GST = net price * gst/100, amount = net + GST.
- For discount_after_gst at item level: amount = (base + base*GST/100) * (1 - discount/100). For percentage discounts, this gives the same final total as discount_before_gst because multiplication is commutative.

6.2 Overall Discount and GST Toggle

- overall_discount is treated as a rupee amount, not a percentage.
- Both _calculate_discount_before_gst and _calculate_discount_after_gst currently do the same thing for overall discount: calculate item GST first, then subtract overall_discount from the final total.
- Therefore the GST method toggle does not actually change total GST for overall discount. If the goal is GST filing, an overall discount before GST should usually be allocated across taxable line values before GST is calculated.

6.3 Rounding and Total

- Purchase total is displayed after item amounts, overall discount, and rounding.
- The UI auto-calculates rounding to the nearest rupee and writes the rounding entry when the field is not focused.
- total_amount_var is the UI source used by save_purchase for total_amount.

6.4 Supplier Due, Credit, Paid

- load_supplier_due reads latest purchases.total_due and purchases.credit_amount and places them into previous_due and previous_credit fields.
- calculate_payment_due displays net amount as purchase_total + previous_due - previous_credit.
- However, calculate_payment_due then calculates final due using purchase_total + previous_due - amount_paid, ignoring previous_credit.
- save_purchase reads only latest total_due as previous_due. It does not read or apply latest credit_amount or previous_credit.
- As a result, supplier credit is displayed but not reliably applied to the next purchase.

6.5 Purchase Save and Account Clear

- save_purchase inserts a purchase row with total_amount, amount_paid, previous_due, due_amount, credit_amount, total_due, bill_cleared, and account_cleared.
- If net_owed = previous_due + total_amount - amount_paid is positive, total_due is net_owed. due_amount is current bill unpaid amount only when current payment is less than current total.
- If current bill is paid but previous due remains, due_amount is zero, bill_cleared is 1, account_cleared is 0.
- If account_cleared is 1, all previous purchases for that supplier are marked account_cleared = 1.
- If account_cleared is 0, the new purchase flow marks all purchases for that supplier account_cleared = 0. This can erase old cleared-history state when a new due purchase is added.

7. Purchase History Flow

7.1 Display and Filters

- Purchase History reads stored database values for due_amount, credit_amount, total_due, account_cleared, and bill_cleared.

- Due Only filter uses `due_amount > 0`. This can miss suppliers whose current purchase bill is cleared but previous supplier balance is still due in `total_due`.
- Rows are tagged using `account_cleared` first, then `bill_cleared`, then `total_due > 0`.
- Summary totals current due/credit by selecting the latest purchase per supplier inside the currently filtered data. If the filter excludes a supplier latest row, the summary can show a historical balance rather than current balance.

7.2 Exports

- Purchase Register exports total, paid, due, credit, and `total_due` from purchases.
- Supplier Due Report exports every purchase row with `total_due > 0` and `account_cleared = 0`, not just latest row per supplier. This can duplicate running balances if multiple open rows exist.
- GST Purchase Report exports GST percent, qty, rate, and amount. It does not export taxable value, CGST, SGST, IGST, allocated discount, or input tax amount separately, so it is not GST-filing-ready.

7.3 Purchase Edit

- Purchase edit loads stored `purchase_items` and stored purchase fields into a `PurchasePage` instance.
- The edit save path calculates `total_amount` as `sum(item.amount) + rounding`. It does not subtract `overall_discount` even though `overall_discount` is saved and displayed. Existing purchases with overall discount can be saved at a higher total after editing.
- The edit save path reads `previous_credit` but does not use it in `due/credit/total_due` calculation.
- The edit save path deletes and reinserts `purchase_items` but does not adjust medicine stock for changed quantities/items. Stock can become wrong after purchase edit.
- The edit save path preserves old account-cleared periods better than new purchase save, but still does not recalculate later supplier balances in chronological order.

7.4 Purchase Delete

- Deleting a purchase subtracts `qty + free_qty` from medicine stock, then deletes `purchase_items` and purchases rows.
- It does not recalculate later supplier balances or `account_cleared` flags after deletion.
- Deleting a historical purchase can leave subsequent supplier `total_due/account_cleared` values stale.

8. Highest-Priority Problems to Fix in a Rebuild

Priority	Problem	Impact
P0	Sales GST is not saved historically	GST filing and old bill GST are unreliable.
P0	Customer and supplier credits are displayed but not consistently applied to later bills	Credit balances can be ignored, causing wrong payable/receivable amounts.
P0	Sales edit treats stored rupee discount as percentage	Edited sales can become dramatically wrong.
P0	Purchase edit omits overall discount and does not adjust stock	Edited purchases can corrupt both accounting and stock.
P1	Account-cleared flags are maintained by several duplicated code paths plus triggers	Rows can drift after edits/deletes and historical cleared status can be erased.
P1	Profit uses current medicine MRP/rate instead of historical sale and purchase cost	Profit reports are not business-accurate.
P1	Overall discount GST method toggle does not change GST calculation	Purchase GST report cannot support filing-level logic.
P2	Due exports can list multiple running-balance rows per party	Manual reports can double-count dues.

9. Recommended Rebuild Principle

Do not rebuild by patching each screen separately. The safer design is to create one accounting calculation layer and make every screen call it. The database should store immutable invoice facts and ledger events; reports should calculate from those facts, not from mutable running totals in many rows.

Recommended Source of Truth

- Invoice header: one row per sale/purchase with immutable totals, payment status, and party id.
- Invoice items: one row per item with frozen rate, discount, taxable value, GST rate, GST amount, total amount, HSN, batch, unit conversion, and cost snapshot where needed.
- Payments: separate rows for cash, online, refund, adjustment, and credit use.
- Party ledger: signed debit/credit entries generated from invoices, payments, returns, and adjustments.
- Stock ledger: signed stock movements generated from purchases, sales, purchase returns, sales returns, corrections, and opening stock.
- Reports: GST, profit, account due, and stock reports read ledgers and item tax facts.

10. GST Filing Rebuild Design

10.1 Sales GST

- Freeze GST rate per sale item at the moment of sale.
- If MRP is GST-inclusive, $\text{line_net_inclusive} = \text{qty} * \text{sale_rate} - \text{item_discount} - \text{allocated_bill_discount} + \text{allocated_rounding}$ as applicable.
- $\text{taxable_value} = \text{line_net_inclusive} * 100 / (100 + \text{gst_rate})$.
- $\text{gst_amount} = \text{line_net_inclusive} - \text{taxable_value}$.
- For local/intra-state sales, split gst_amount into CGST and SGST. For inter-state sales, use IGST if the business later needs it.
- Store taxable_value , cgst_amount , sgst_amount , igst_amount , gst_rate , and line_total on each sale item.

10.2 Purchase GST

- For purchase bills, decide whether supplier rates are GST-exclusive or GST-inclusive. Current code treats purchase rate as GST-exclusive.
- Item discount before GST should reduce taxable value before GST is calculated.
- Overall bill discount before GST should be allocated proportionally across taxable item values and then reduce each line taxable value.
- GST amount should be calculated per line after discount allocation. Do not calculate one GST total and split afterward if mixed GST rates exist.
- Store input taxable value and input GST by line for filing and audit.

10.3 Discount Allocation Formula

- For bill-level rupee discount before GST: $\text{line_share} = \text{line_taxable_before_bill_discount} / \text{total_taxable_before_bill_discount}$.
- $\text{line_bill_discount} = \text{bill_discount} * \text{line_share}$.
- $\text{line_taxable_after_discount} = \text{line_taxable_before_bill_discount} - \text{line_bill_discount}$.
- $\text{line_gst} = \text{line_taxable_after_discount} * \text{gst_rate} / 100$.
- $\text{line_total} = \text{line_taxable_after_discount} + \text{line_gst}$.

10.4 Reports Needed for Filing

- Sales GST summary by date range, GST rate, HSN, taxable value, CGST, SGST, IGST, invoice value.
- Purchase GST summary by date range, supplier GSTIN, GST rate, HSN, taxable value, input CGST, input SGST, input IGST.

- Credit/debit note register for sale returns and purchase returns.
- Invoice-wise audit report that ties item tax totals to invoice total, rounding, and payments.

11. Profit Calculation Rebuild Design

Profit should be based on actual historical sale revenue excluding GST and actual historical cost excluding GST. The current formula uses current medicine MRP/rate and ignores discounts, GST, and batch cost changes.

Recommended Profit Formula

- sale_revenue_ex_gst = sale item taxable value after discounts.
- cost_ex_gst = quantity sold * batch purchase cost excluding GST.
- gross_profit = sale_revenue_ex_gst - cost_ex_gst.
- profit_after_returns = gross_profit - returned_profit_reversal.
- If item-level batch link is unavailable, choose FIFO or weighted-average costing and store the selected cost snapshot on sale_items.

Data to Store for Profit

- sale_items.purchase_item_id or cost_batch_id when possible.
- sale_items.cost_rate_ex_gst snapshot.
- sale_items.sale_taxable_value and sale GST amounts.
- return rows linked to original sale_items/purchase_items to reverse profit and GST correctly.

12. Account Cleared and Credit Rebuild Design

Running balances should not be hand-written into every invoice row as the primary truth. Use a party ledger and calculate current balance from ledger entries. Keep invoice-level cleared state separately from account-level balance.

Party Ledger Signs

- For customers: sale invoice is debit; payment is credit; sale return/refund depends on refund vs credit note.
- For suppliers: purchase invoice is credit/payable; supplier payment is debit; purchase return/debit note reduces payable.
- Current balance > 0 means due. Current balance = 0 means clear. Current balance < 0 means credit/advance.

Invoice Clear vs Account Clear

- invoice_cleared should mean that a specific invoice has been fully paid or settled by credit/return.
- account_cleared should be a derived party status at a point in time, not a flag updated across many old rows.
- If a historical audit trail is needed, store account_clear_events with party_id, cleared_at_datetime, balance, and source transaction id.
- Do not mutate old invoice rows just to display account clear color. Display colors can be derived from current ledger status.

13. Discount Rebuild Rules

- Clearly separate item discount percent, item discount amount, bill discount percent, and bill discount amount.
- Store both the user input type and the calculated rupee amount.
- For sales, decide whether customer bill discount reduces GST taxable value or is treated as after-tax discount. For GST filing, this must be explicit.
- For purchases, remove or redefine the current GST method toggle because both methods currently produce the same overall-discount behavior.
- Always calculate totals through one pure function, then save the returned result. Never re-derive totals differently in edit screens.

14. Future Sales Return and Purchase Return Design

Sales Return

- Create sale_returns and sale_return_items tables linked to original sale and sale_items.
- Return quantity cannot exceed sold quantity minus previous returns.
- Stock movement: returned quantity increases stock for the original batch or a return holding location.
- Accounting: reduce customer balance or create customer credit/refund payment.
- GST: create a credit note reversing taxable value and GST for returned items.
- Profit: reverse original profit using the original cost snapshot.

Purchase Return

- Create purchase_returns and purchase_return_items linked to original purchase and purchase_items.
- Return quantity cannot exceed available purchased quantity minus sold/reserved quantity rules, depending on operational policy.
- Stock movement: returned quantity reduces stock.
- Accounting: reduce supplier payable or create supplier debit/credit note.
- GST: reverse input tax for returned items.
- Costing: adjust available batch cost layers and prevent negative stock/value.

15. Migration Plan If You Remove and Rebuild Logic

- Step 1: Make a full database backup and keep an untouched copy of the current application folder.
- Step 2: Freeze current accounting behavior. Do not make more manual fixes in multiple UI files while the rebuild is underway.
- Step 3: Create a calculation specification document with exact formulas for sale, purchase, discount, GST, rounding, payments, credits, and returns.
- Step 4: Build one pure calculation service. It should accept invoice inputs and return item totals, header totals, ledger entries, stock movements, and validation messages.
- Step 5: Add new tables beside old tables: invoice items with tax fields, payments, party ledger, stock ledger, return tables, and audit events.
- Step 6: Write migration scripts that read old sales/purchases and populate new tables. Mark migrated GST/profit values as best-effort where historical GST or item discount is missing.
- Step 7: Replace Sales, Purchase, Sales History edit, and Purchase History edit so all of them call the same calculation service.
- Step 8: Remove calculation triggers or reduce them to validation only. Do not duplicate business calculations in triggers and UI code.
- Step 9: Rebuild reports from the new item tax tables and ledgers.
- Step 10: Run test cases below before using the rebuilt system for live GST or financial reporting.

16. Test Matrix for the Rebuilt Logic

Case	Expected result
Sale paid exactly	due_amount 0, invoice clear true, account balance unchanged except sale/payment net zero.
Sale partially paid	current invoice balance positive, customer ledger due positive.
Customer overpayment	invoice clear true, account balance credit negative, next sale uses credit automatically.

Case	Expected result
Sale with item discount and bill discount	stored item discount, allocated bill discount, taxable value and GST match invoice total.
Mixed GST sale	GST summary grouped by each GST rate and HSN, not one blended rate.
Purchase with item discount	input taxable value reduced before GST if configured as before-GST discount.
Purchase with overall discount before GST	bill discount allocated across lines and GST reduced accordingly.
Supplier credit then next purchase	credit offsets next payable before new due is created.
Edit sale discount	stored amount/percent round-trips without changing meaning.
Edit purchase with overall discount	total remains correct and stock adjusts only by quantity delta.
Delete historical invoice	later balances are recalculated or ledger void entries keep audit trail consistent.
Sales return	stock increases, GST credit note created, profit reversed.
Purchase return	stock decreases, input GST reversed, supplier balance adjusted.

17. Suggested New Table Sketch

This is a conceptual sketch, not code to apply directly.

parties: id, type(customer/supplier), name, phone, address, gstin, dl_numbers

invoices: id, type(sale/purchase), party_id, invoice_no, invoice_date, gross_amount, item_discount_amount, bill_discount_amount, taxable_total, cgst_total, sgst_total, igst_total, rounding, grand_total, status

invoice_items: id, invoice_id, medicine_id, batch_no, qty, free_qty, unit_factor, rate, mrp, discount_type, discount_input, discount_amount, bill_discount_allocated, taxable_value, gst_rate, cgst_amount, sgst_amount, igst_amount, line_total, hsn_code, cost_rate_snapshot

payments: id, party_id, invoice_id nullable, type(cash/online/refund/adjustment/credit_use), amount, paid_at, reference

party_ledger: id, party_id, source_type, source_id, debit, credit, entry_date, notes

stock_ledger: id, medicine_id, batch_no, source_type, source_id, qty_in, qty_out, unit_factor, cost_rate, created_at

sale_returns / sale_return_items and purchase_returns / purchase_return_items linked to original invoice_items

18. Final Recommendation

The current system can continue to display and save simple bills, but it should not be treated as GST-filing-ready or profit-accurate. The safest path is to centralize calculations, store historical tax/cost facts per item, and move party balances into a ledger. The biggest immediate risks to avoid before a rebuild are editing old sales with discount, editing purchases with overall discount, and relying on sales GST exports.